Breaking and Non-Breaking Spaces in SQL, Spark SQL, and PySpark

Practical guide for data engineering and ETL pipelines

1. Quick summary

A **breaking space** is the ordinary space character (`U+0020`). Text layout can wrap at it. A **non-breaking space (NBSP)** is a different character (`U+00A0`) that normally prevents a line break at that position. They can look identical, but they are not the same character.

In SQL and Spark, the important issue is character identity: an ordinary space and NBSP may not compare equal, may prevent a join from matching, and may survive a default `TRIM`. Exact comparison behavior can depend on the engine, data type, collation, and version.

Common cleanup pattern: replace NBSP with an ordinary space, then trim the result. In Databricks/Spark SQL, use a supported way to construct NBSP (for example, `CHR(160)` where supported, or `DECODE(UNHEX('C2A0'), 'UTF-8')` for UTF-8 bytes). In PySpark, the clearest approach is usually `regexp_replace` with Python's `"\u00A0"` literal.

2. Breaking space vs. non-breaking space

Property	Ordinary/breaking space	Non-breaking space
Unicode	U+0020	U+00A0
UTF-8 bytes	20	C2 A0
Appearance	Normal blank	Usually looks like a normal blank
Text wrapping	Usually permits a line break	Normally prevents a line break
Same character?	No	No
Default TRIM behavior	Commonly removed at edges	Not reliably removed
Typical sources	Keyboard input, normal text	HTML , web pages, copied documents, spreadsheets

“Breaking” describes text layout, not SQL parsing. SQL does not split a string just because it contains an ordinary space. NBSP is also distinct from tabs, newlines, narrow NBSP (`U+202F`), and zero-width space (`U+200B`).

3. Why this matters in SQL

3.1 Filters

A predicate such as `WHERE customer_name = 'Ravi Kumar'` may not match a stored value whose separator is NBSP. The strings look the same but contain different code points.

3.2 Joins and MERGE operations

If one table contains `Ravi<U+00A0>Kumar` and another contains `Ravi<U+0020>Kumar`, a join on the text value may fail. In an ETL `MERGE`, a failed match can cause a new row to be inserted instead of updating an existing row, depending on the merge logic.

Prefer stable identifiers (such as `customer_id`) over names as business keys. If text matching is required, normalize the key consistently on both sides.

3.3 Grouping and deduplication

`GROUP BY`, `DISTINCT`, and window-based deduplication may treat values containing different whitespace characters as different groups/keys. Collations can affect comparisons, so validate behavior in your actual engine.

3.4 `TRIM` is not a universal whitespace cleaner

Default `TRIM()` usually removes ordinary spaces at the start and end. It does not generally remove an NBSP embedded between words, and it may not remove NBSP at the edges unless you explicitly replace or trim that character.

3.5 Counts and data quality

Unnormalized text can inflate distinct counts, break uniqueness checks, and cause unmatched records between Bronze and Silver datasets.

4. Detect NBSP before cleaning

4.1 Databricks / Spark SQL

Where supported by the runtime, `CHR(160)` can represent NBSP:

```
SELECT
  'Ravi Kumar' AS normal_value,
  CONCAT('Ravi', CHR(160), 'Kumar') AS nbsp_value,
  'Ravi Kumar' = CONCAT('Ravi', CHR(160), 'Kumar') AS is_equal;
```

To find rows containing NBSP:

```
SELECT *
FROM customers
WHERE INSTR(customer_name, CHR(160)) > 0;
```

If `CHR(160)` is not available in your Spark/Databricks runtime, use the UTF-8 byte sequence:

```
-- UTF-8 bytes C2 A0 represent U+00A0
SELECT DECODE(UNHEX('C2A0'), 'UTF-8') AS nbsp_character;
```

Then use that expression as the search/replacement string if your runtime supports the expression in that context.

4.2 MySQL

```
SELECT *
FROM customers
WHERE LOCATE(
  CONVERT(0xC2A0 USING utf8mb4),
  customer_name
) > 0;
```

4.3 SQL Server

```
SELECT *
FROM customers
WHERE CHARINDEX(NCHAR(160), customer_name) > 0;
```

4.4 PostgreSQL

```
SELECT *
FROM customers
WHERE POSITION(U&'\00A0' IN customer_name) > 0;
```

4.5 Inspect bytes or code points

When values look identical, inspect their encoded representation where your engine provides a suitable function. In Spark SQL, `HEX(ENCODE(value, 'UTF-8'))` can help inspect UTF-8 bytes:

```
SELECT
  customer_name,
  HEX(ENCODE(customer_name, 'UTF-8')) AS utf8_hex
FROM customers;
```

For the separator, an ordinary space has byte `20`; NBSP has bytes `C2A0`. `HEX` output may include the bytes for the entire string, not just the space.

5. Spark SQL: what happens and how to clean

Spark SQL evaluates string functions on the characters in a string. An NBSP is not automatically converted to an ordinary space just because it looks like one. `TRIM` and equality behavior should not be assumed to normalize it.

5.1 Create a test dataset

```
WITH sample AS (
  SELECT 'Ravi Kumar' AS name
```

```
  UNION ALL
  SELECT CONCAT('Ravi', CHR(160), 'Kumar')
  UNION ALL
  SELECT ' Ravi Kumar '
)
SELECT
  name,
  LENGTH(name) AS original_length,
  HEX(ENCODE(name, 'UTF-8')) AS utf8_hex,
  TRIM(REPLACE(name, CHR(160), ' ')) AS cleaned_name
FROM sample;
```

If `CHR(160)` is unavailable, replace it with a supported NBSP expression such as `DECODE(UNHEX('C2A0'), 'UTF-8')` and test it in your runtime.

The ordinary-space and NBSP versions can have the same character length but different UTF-8 bytes. That is why length alone is not a reliable detector.

5.2 Replace NBSP, then trim

```
SELECT
  TRIM(REPLACE(customer_name, CHR(160), ' ')) AS cleaned_name
FROM customers;
```

This:

1. replaces NBSP with an ordinary space;

2. trims ordinary spaces from the beginning and end.

It intentionally preserves a separator between words.

5.3 Also collapse repeated ordinary spaces

```
SELECT
  TRIM(
    REGEXP_REPLACE(
      REPLACE(customer_name, CHR(160), ' '),
      ' +',
      ' '
    )
  ) AS cleaned_name
FROM customers;
```

This collapses runs of ordinary ASCII spaces. It does not cover every Unicode whitespace character. Extend the rule only after identifying what the source actually contains.

5.4 Update a Delta table carefully

```
UPDATE customers
SET customer_name =
  TRIM(REPLACE(customer_name, CHR(160), ' '))
WHERE INSTR(customer_name, CHR(160)) > 0;
```

Before updating production data, preview affected rows, check duplicates created by normalization, and follow your team's backup/rollback process. Prefer a cleaned derived column or Silver transformation when retaining the original source value is important.

6. PySpark: DataFrame API solution

In PySpark, Python can express NBSP directly as `"\u00A0"`. Spark still performs the transformation on the DataFrame; Python is not looping over every row.

6.1 Build a sample DataFrame

```
from pyspark.sql import functions as F

sample = spark.createDataFrame(
    [
        ("Ravi Kumar",),
        ("Ravi\u00A0Kumar",),  # NBSP between words
        (" Ravi Kumar ",),
        ("Ravi   Kumar",),
    ],
    ["customer_name"]
)
```

```
sample.show(truncate=False)
```

6.2 Detect rows containing NBSP

```
from pyspark.sql import functions as F

nbsp = "\u00A0"

sample.filter(
    F.instr(F.col("customer_name"), nbsp) > 0
).show(truncate=False)
```

Alternative using a regular expression:

```
sample.filter(
    F.col("customer_name").rlike("\u00A0")
).show(truncate=False)
```

For literal character searches, `instr` is straightforward. `rlike` interprets its argument as a regular expression.

6.3 Replace NBSP and trim

```
from pyspark.sql import functions as F

nbsp = "\u00A0"

cleaned = sample.withColumn(
    "customer_name_clean",
    F.trim(
        F.regexp_replace(
            F.col("customer_name"),
            nbsp,
            " "
        )
    )
)

cleaned.show(truncate=False)
```

This changes NBSP to an ordinary space, then trims leading/trailing ordinary spaces.

6.4 Collapse repeated spaces too

```
from pyspark.sql import functions as F

nbsp = "\u00A0"

cleaned = sample.withColumn(
    "customer_name_clean",
    F.trim(
        F.regexp_replace(
            F.regexp_replace(
                F.col("customer_name"),
                nbsp,
                " "
            ),
            " +",
            " "
        )
    )
)
```

The inner `regexp_replace` handles NBSP. The outer one collapses consecutive ASCII spaces. `trim` removes ASCII spaces at the edges.

6.5 Clean multiple selected columns

Use a controlled list of text columns rather than blindly modifying every string field:

```
from pyspark.sql import functions as F

nbsp = "\u00A0"
text_columns = ["customer_name", "city", "address"]

df_clean = df
for c in text_columns:
    df_clean = df_clean.withColumn(
        c,
        F.trim(
            F.regexp_replace(F.col(c), nbsp, " ")
```

```
        )
    )
```

This is a driver-side loop that builds a DataFrame plan; Spark executes the transformations when an action runs. For very wide schemas, generate the expressions with `select` if that makes the transformation easier to maintain.

6.6 Normalize both sides of a join

```
from pyspark.sql import functions as F

nbsp = "\u00A0"

left_clean = left_df.withColumn(
    "name_key",
    F.trim(F.regexp_replace("customer_name", nbsp, " "))
)

right_clean = right_df.withColumn(
    "name_key",
    F.trim(F.regexp_replace("customer_name", nbsp, " "))
)

joined = left_clean.join(right_clean, on="name_key", how="inner")
```

Do not normalize only one side. Use a stable ID whenever possible. If collapsing repeated spaces is part of your rule, apply that same rule to both sides.

7. Spark SQL vs. PySpark

Topic	Spark SQL	PySpark
How you express the transformation	SQL string functions	DataFrame functions
NBSP representation	CHR(160) where supported, or a UTF-8 expression	Python literal "\u00A0"
Detect	INSTR(col, CHR(160)) > 0	F.instr(F.col(col), "\u00A0") > 0
Replace	REPLACE(col, CHR(160), ' ')	F.regexp_replace(F.col(col), "\u00A0", " ")
Trim	TRIM(col)	F.trim(F.col(col))
Collapse repeated ASCII spaces	REGEXP_REPLACE(col, ' +', ' ')	F.regexp_replace(col, " +", " ")
Execution	Spark SQL engine	Spark SQL engine
Main difference	SQL syntax	Python API builds the Spark query plan

PySpark is not a separate whitespace engine. For these DataFrame operations, Spark builds and executes a distributed query plan. Both approaches should produce equivalent results when they express the same rules and run on the same Spark version/configuration.

8. Other whitespace characters

Character	Unicode	Potential source
Ordinary space	U+0020	Keyboard
NBSP	U+00A0	HTML, copied text, spreadsheets
Tab	U+0009	Text files, spreadsheets
Line feed	U+000A	Multiline text
Carriage return	U+000D	Text file line endings
Narrow NBSP	U+202F	Formatted numbers, international text
Zero-width space	U+200B	Web content, copied text

Do not treat all these characters as interchangeable. A newline may be meaningful in an address or description. A zero-width space may need removal rather than replacement. Define a rule per column and use case.

For example, to replace NBSP and narrow NBSP in PySpark:

```
nbsp_and_narrow_nbsp = "[\u00A0\u202F]"
```

```
df_clean = df.withColumn(
    "customer_name_clean",
    F.trim(
        F.regexp_replace(
            F.col("customer_name"),
            nbsp_and_narrow_nbsp,
            " "
        )
    )
)
```

This is a regex character class containing those two characters; it is not a universal Unicode-whitespace solution.

9. Data engineering recommendations: Bronze → Silver → Gold

Bronze

• Preserve raw source values for traceability where practical.

• Identify affected columns and source systems.

• Measure how many records contain unexpected whitespace.

Silver

• Apply explicit, documented normalization rules.

• Use the same rules for both sides of joins and for incremental/full loads.

• Validate null counts, row counts, duplicate business keys, and unmatched records after cleaning.

• Keep the raw value in a separate column if auditability matters.

Gold

• Use standardized fields for reporting and analytics.

• Avoid using names as keys when stable IDs exist.

• Monitor data-quality metrics so the issue does not silently return.

Useful validation checks

Check how many records contain NBSP in Spark SQL:

```
SELECT COUNT(*) AS rows_with_nbsp
FROM customers
WHERE INSTR(customer_name, CHR(160)) > 0;
```

Check whether normalization creates duplicate names:

```
WITH normalized AS (
  SELECT
    TRIM(REPLACE(customer_name, CHR(160), ' ')) AS normalized_name
  FROM customers
)
SELECT normalized_name, COUNT(*) AS row_count
FROM normalized
GROUP BY normalized_name
HAVING COUNT(*) > 1;
```

This finds duplicate normalized names, not necessarily duplicate customers. Validate against the real business key.

10. Common mistakes

1. **Assuming `TRIM` removes every whitespace character.** It does not necessarily remove NBSP or all Unicode whitespace.

2. **Deleting NBSP rather than replacing it.** `Ravi Kumar` could become `RaviKumar`.

3. **Cleaning only one side of a join.** Both sides must follow the same normalization contract.

4. **Assuming length will expose the issue.** U+0020 and U+00A0 are each one Unicode character in this example, but have different UTF-8 bytes.

5. **Running an update without checking collisions.** Different values can become identical after normalization.

6. **Applying a broad regex to every column.** Whitespace can be meaningful in some fields.

7. **Assuming every Spark runtime supports the same character-construction function.** Verify `CHR(160)` in your runtime; use a supported alternative if necessary.

11. Interview-ready explanation

Question: Why can SQL joins fail when values look identical?

Because strings can contain different Unicode characters that render similarly. For example, an ordinary space is `U+0020`, while a non-breaking space is `U+00A0`. If one table contains one character and the other table contains the other, equality and join matching may fail depending on the engine and collation. A robust solution is to normalize the relevant key consistently on both sides, validate duplicates created by normalization, and prefer stable IDs over names.

Question: How would you fix NBSP in Databricks?

In Spark SQL, replace NBSP with an ordinary space and trim the result, for example `TRIM(REPLACE(customer_name, CHR(160), ' '))` where `CHR` is supported. In PySpark, use `F.regexp_replace(F.col("customer_name"), "\u00A0", " ")`, followed by `F.trim`. Test the runtime behavior, apply the same rule to join keys on both sides, and validate the output.

12. Final checklist

• [ ] Confirm which whitespace character is present.

• [ ] Detect NBSP using INSTR, CHARINDEX, LOCATE, or POSITION, depending on the database.

• [ ] Replace NBSP with an ordinary space when that is semantically correct.

• [ ] Trim edges; collapse repeated spaces only if required.

• [ ] Apply identical rules to both sides of joins and merge keys.

• [ ] Check for duplicate keys and row-count changes after cleaning.

• [ ] Preserve raw values where traceability is needed.

• [ ] Add data-quality checks to prevent recurrence.

Default Databricks/Spark SQL starting point:

```
TRIM(REPLACE(your_column, CHR(160), ' '))
```

Default PySpark starting point:

```
F.trim(F.regexp_replace(F.col("your_column"), "\u00A0", " "))
```

Always verify the character-construction syntax and behavior on the Spark/Databricks runtime you actually use.